Task 3

Mentor Review Packet

Cryptographic Session Security: HttpOnly Cookies and JWT Defense

Prepared for Datheon internship mentors and project reviewers

05 November 2026

Review scope and safety notice

This is a loopback-only security lab with synthetic accounts and a lab-only password. Never reuse the fixtures, signing key, or cookies in a real service.

Contents

1 Reviewer at a glance 1
2 Scope and security objectives 1
3 Secure session workflow 1
4 Representative security outputs 2
5 Visual evidence for reviewers 2
6 Verification summary 3
7 Mentor review conclusion 4

1 Reviewer at a glance

Task 3 replaces browser-readable authentication storage with a cryptographically signed session cookie. The implementation issues an HMAC-SHA256 JWT only in an `HttpOnly`, `Secure`, `SameSite=Lax` cookie; verifies its claims and signature on protected requests; rotates the session identifier; expires sessions; supports logout revocation; and throttles login and protected reads.

Verified implementation The automated suite passed **28 tests** and the TypeScript check passed. Coverage includes cookie flags, token absence from JSON and Web Storage, tamper/expiry rejection, rotation and replay rejection, absolute lifetime, logout, BOLA ownership, rate-limit headers, `429` handling, and sliding-window expiry.

What reviewers can verify The browser entry point is a normal sign-in page. The security properties are verified at the HTTP and integration-test layers: the token is delivered through `Set-Cookie`, never returned in JSON, and the old token is rejected after rotation or logout.

2 Scope and security objectives

Objective	Implementation evidence
Storage defense	No authentication token or profile data in `localStorage` or `sessionStorage`; cookie is `HttpOnly`
Cryptographic integrity	HS256 JWT with allowed algorithm, issuer, audience, subject, session ID, JTI, and expiry validation
Lifetime	15-minute access token, eight-hour absolute session family limit
Rotation	New JTI replaces the old token; replay of the prior token is rejected
Revocation	Logout revokes the session family and clears the cookie
Throttling	Five login attempts/minute/IP and ten protected reads/minute/IP+session

3 Secure session workflow

1. Login

Verify the salted password hash.

→

2. Issue

Set the signed JWT in an HttpOnly cookie, not JSON.

→

3. Verify

Validate claims, signature, session registry, and expiry.

→

4. Rotate

Replace JTI and reject replay of the previous token.

4 Representative security outputs

The following outputs summarize the verified behavior without printing any secret, cookie value, or signing key:

```
POST /api/session                          → 200 OK
Set-Cookie: northstar_session=[redacted]; HttpOnly; Secure; SameSite=Lax
JSON body: account details only; no JWT token

tampered or expired cookie                  → 401 Unauthorized
replayed prior cookie after rotation        → 401 Unauthorized
logout                                     → cookie cleared; protected request → 401
login burst: attempts 1–5                   → allowed
login burst: attempt 6                      → 429 Too Many Requests
X-RateLimit-Limit / Remaining / Reset       → present on protected routes
Retry-After                                 → present on 429 responses
```

Why HttpOnly matters Web Storage is readable by same-origin JavaScript. If an XSS flaw executes in the page, a bearer token stored there can be copied. `HttpOnly` prevents page JavaScript from reading the session cookie, while CSP, output encoding, safe DOM APIs, and CSRF protections remain necessary defense-in-depth.

5 Visual evidence for reviewers

The browser capture below shows the secure-session portal at its local entry point. The form is intentionally ordinary; the security control is the server-side cookie pipeline after sign-in, not a visible token or browser-storage value.

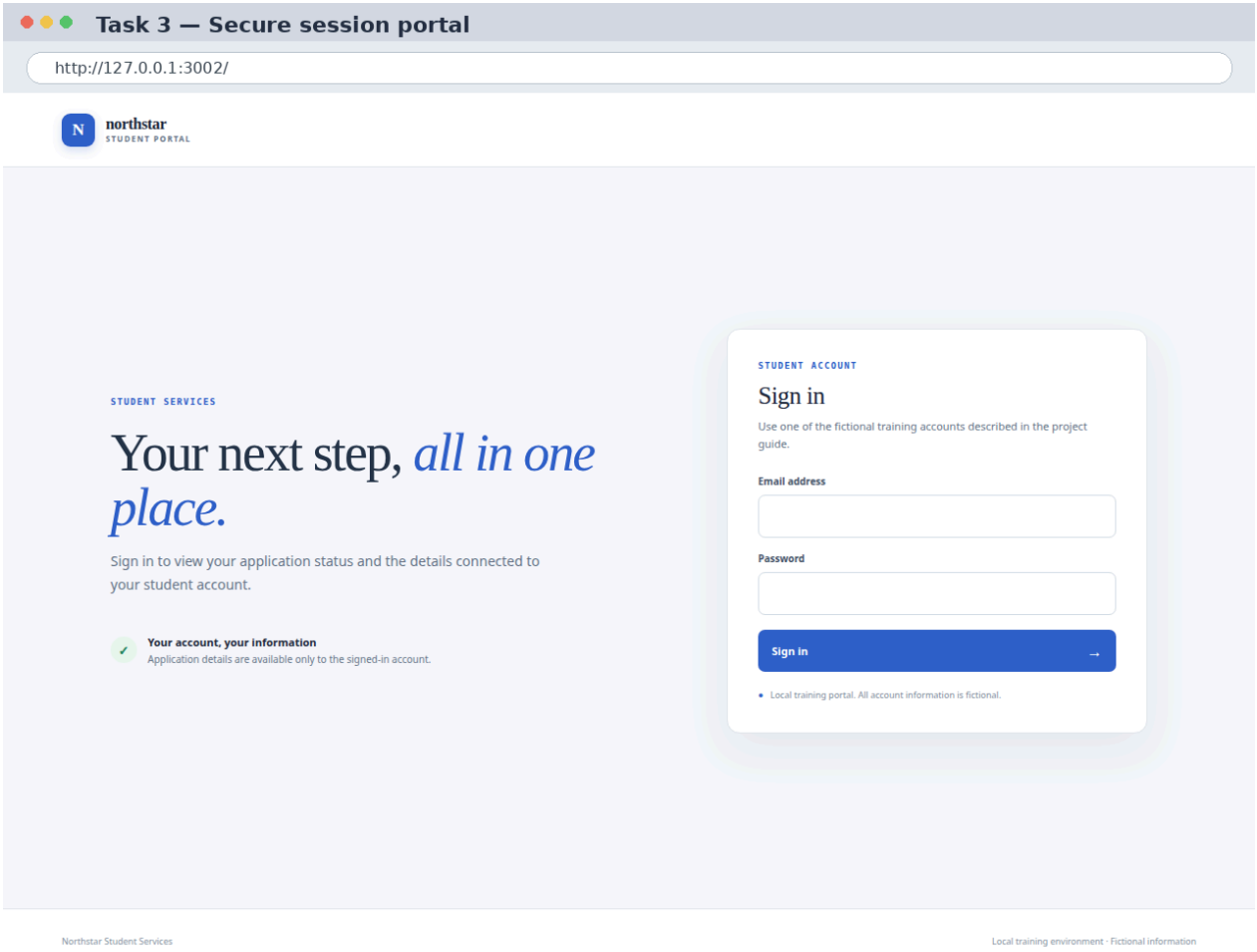


图 1 Browser view of the Task 3 secure-session entry point at `127.0.0.1:3002/`.

Visual evidence interpretation A reviewer should not expect a JWT to appear in this image or in the sign-in JSON. The relevant evidence is the server-set cookie flags and the automated tests that prove no token is placed in Web Storage, old JTIs are rejected, and rate-limit headers are emitted.

6 Verification summary

Check	Result	Coverage
Vitest/Supertest security suite	28 passed	Cookie, JWT, rotation, replay, expiry, logout, BOLA, throttling
TypeScript check	Passed	Strict compile-time verification
Process model	Documented	Single-process in-memory registry and limiter for the lab
Production boundary	Not production-ready	Requires reviewed shared session store and deployment-specific CSRF/TLS policy

7 Mentor review conclusion

Task 3 demonstrates a layered session-security defense: protected cookie transport, cryptographic verification, bounded lifetimes, rotation, revocation, and abuse throttling. The project also retains the Task 2 ownership check so secure authentication does not become a substitute for object-level authorization.

Documentation note: This packet was assembled with AI assistance from the supplied Task 3 source files. Secret values, cookie values, and private keys are intentionally redacted.